

Operating systems

Lecture #1

Content Lecture#1

01 What is an Operating System?

02 Why do we need an OS?

03 System Boot

04 Computer-System Architecture

05 Operating System Concepts

06 Multitasking, Multiprogramming, Multiprocessing

07 Protection

08 Dual-mode of Operation

09 Operating Systems around us



Lets take a Break

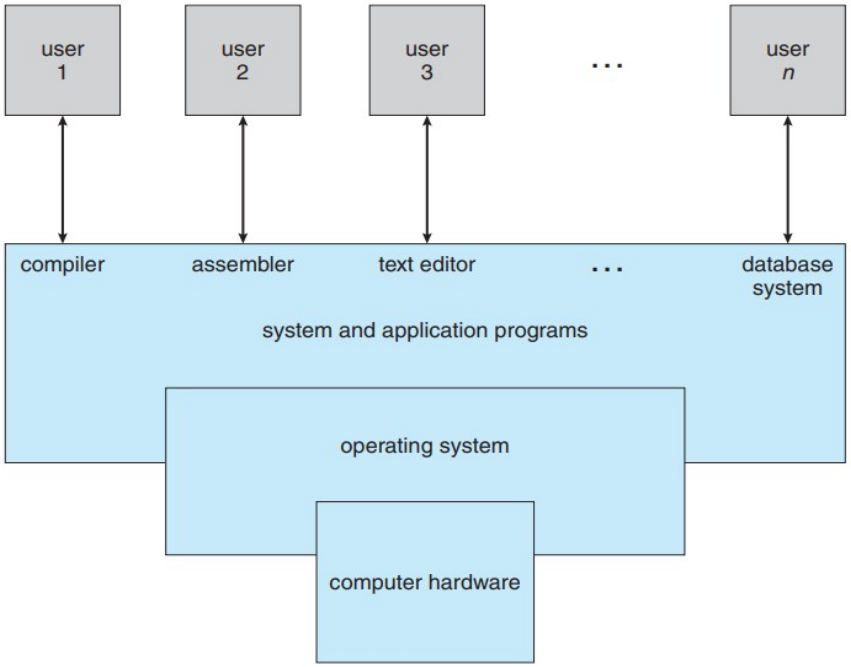
- Silberschatz, Galvin and Gagne, Operating System Concepts, 9th Edition.
- Operating Systems Internals and Design Principles by Willam Stallings 5th edition
- Tanenbaum, Andrew S., Modern Operating Systems, 3rd Edition, Prentice Hall.



Operating System

An operating system is a program that manages a computer’s hardware. It also provides a basis for application programs and acts as an intermediary between the computer user and the computer hardware.

What is Operating System

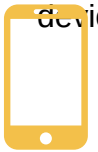


Abstract view of the components of a computer system.



Hardware

the central processing unit (CPU), the memory, and the input/output (I/O) devices



Moore's Law

the number of transistors on an integrated circuit would double every eighteen months,.



Application programs

You can simply impress your audience and add a unique zing.



Resource allocator

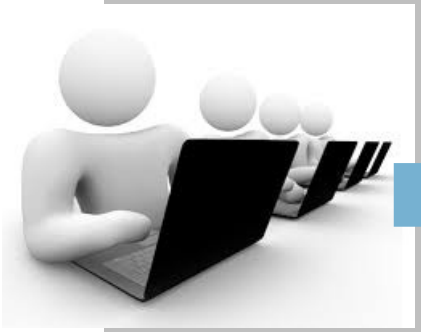
A computer system has many resources that may be required to solve a problem:



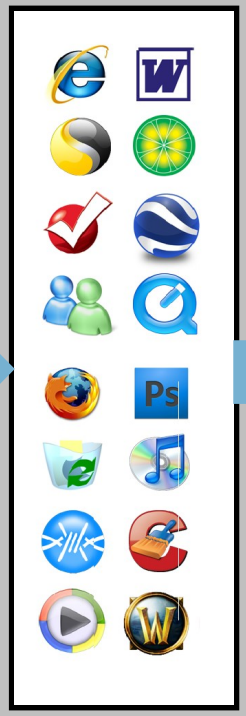
Control program

manages the execution of user programs to prevent errors and improper use of the computer.

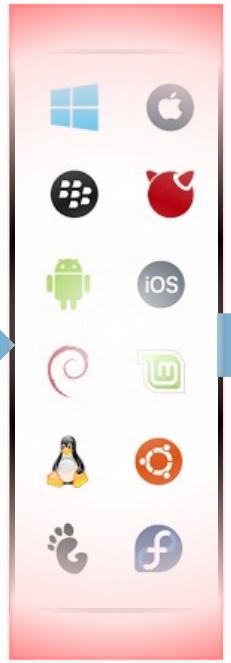
What/Why/How of Operating System



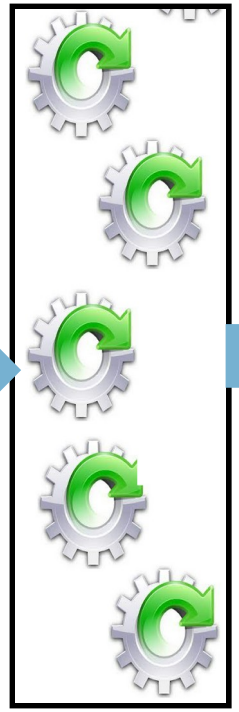
System Users



Application Software



Operating System

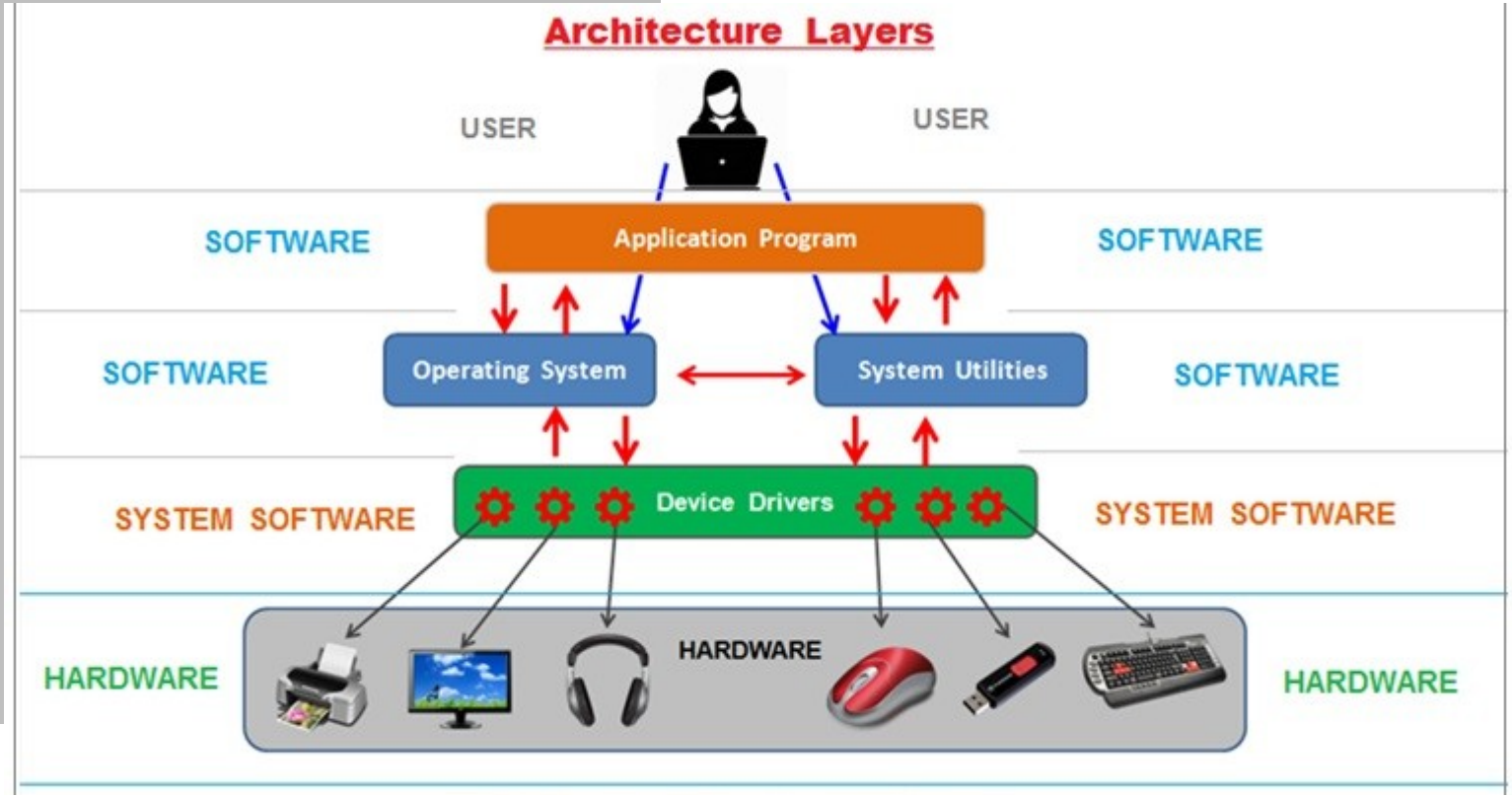


System Program



Hardware

What/Why/How of Operating System



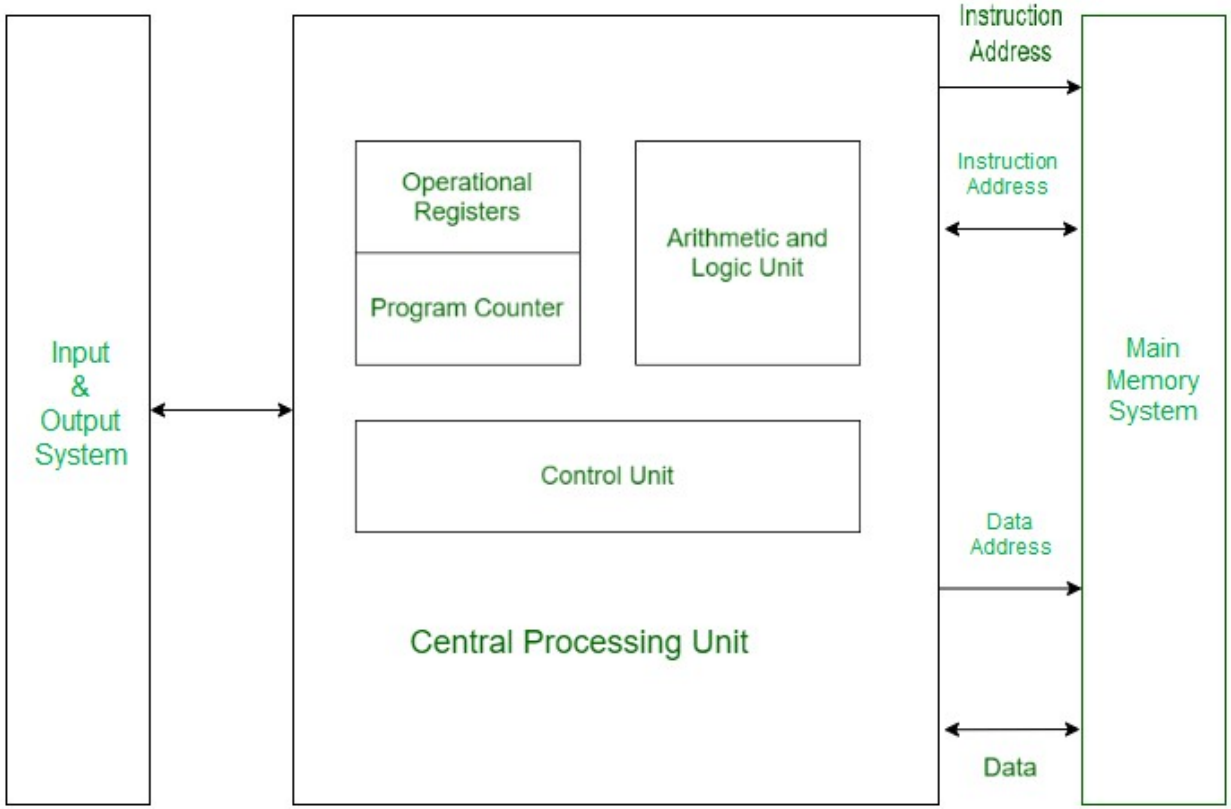
What/Why/How of Operating System

- **System Programs**
  - They are used to manage operations of computer and to control its resources
  - Oriented to the needs of hardware and facilitate the development and running of applications
  - Examples:
    - Compilers (C Language Compiler)
    - Editors (DOS Editor)
    - Shell (C Shell)
    - Interpreter (VB Interpreter)

- **Application Programs**
  - They are used to solve user's problems
  - AP have specific processing's capabilities required for user's applications
  - Examples:
    - Word processor (Microsoft Word, Abi Word)
    - Spread sheet (Microsoft Excel)
    - Presentation tools (MS PowerPoint, ***K***Presenter)
    - Inventory control system
    - Library management system

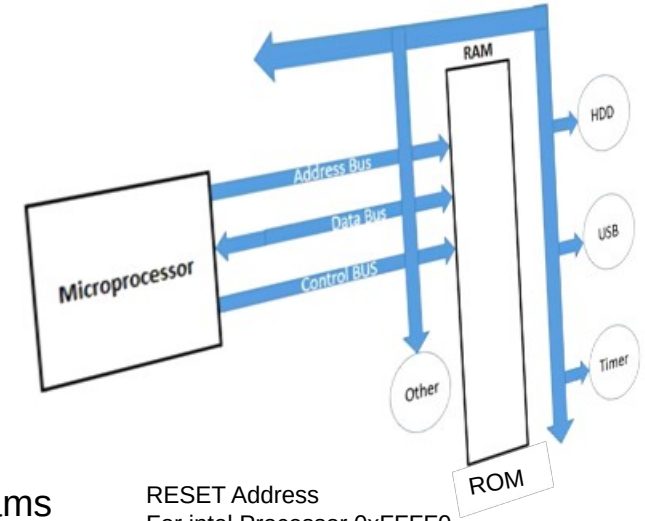
- **Operating Systems**
  - OS is an integrated set of programs used to manage the computer resources and its overall operations
  - Provide a layer of services which manage the resources of the hardware and permit the user to drive the system
  - Provide special hardware services in absence of hardware facility using special software routines

Computer Architecture



Computer Organization

- CPU reads all instructions from RAM
- RAM is empty initially
- Programs are residing on Hard disk
- So initial instructions are read from
- RESET Address which is the address of ROM
- ROM stores the program known as BIOS
- This program loads the first sector of bootable device to 0x07C00(intel)(512 Bytes)

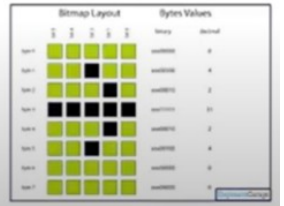


RESET Address
For intel Processor 0xFFFF0

- So after the first sector there is utility to load other programs
  - Provide user interface
  - Provide Programmer interface

BIOS proceeds to load the **MBR** **(Master Boot Record)** from the bootable device into RAM. The MBR consists of 512 or more bytes located at the very beginning sector of the bootable device (which can be an HDD, an SSD, or a flash drive).

- Without Operating System
  - A program has to
    - Read key from keyboard buffer itself
    - Write character patterns directly to video memory
    - Must implement file system
    - Write driver of every hardware it intend to use
    - In nutshell: It will do everything an OS does.



Boot Process

Bootstrap program is loaded at power-up or reboot

- Typically stored in ROM or EEPROM, generally known as **firmware**
- Initializes all aspects of system
- Locates and loads operating system kernel into memory and starts execution
- Sometimes two-step process where **boot block** at fixed location loaded by ROM code, which loads bootstrap loader from disk

Kernel loads and system is then **running**

- The power button is pressed = sends power to the motherboard and other components
- The computer displays a splash screen of the manufacturer's logo, or alternatively text-base specifications of the computer.
- The BIOS access the boot disk, which is typically the same hard disk drive (HDD) or solid-state drive (SSD) that contains the operating system (OS).
- The BIOS loads the bootstrap loader from ROM into the random access memory (RAM). The bootstrap loader is a small program which is used to search for and launch the computer's operating system.
- The BIOS then passes on the work to the boot loader and beings launching the OS.
- Once the boot loader has finished, the operating system takes control of the computer, which is now ready for use.

Evolution of Operating Systems

- In early systems the programs were entered by using a set of switches to define memory address value
- The program was started by setting the program counter to the first instruction word and pressing a start button
- The execution was progressed by displaying light on console
- The first step towards improvement was to reduce human involvement in this process
- Offline input devices like cards or paper tape reader were introduced later
- A program loader was established in Computer memory to read program from input device and setup it in memory

Evolution of Operating Systems

- Around 1960, a new computer called Atlas was developed with the requirements of an operating system in mind
- Batch processing was applied to submit jobs in batches to the computer
- Multiprogramming was introduced to run several programs at the same time
  - The processor should be kept busy for most of the time by switching its attention from one program to another
- Fist time sharing system (support multiple interactive users) was developed at MIT
- Mini Computer took place having multi-user OS
- Spooling (Simultaneous Peripheral Operation On-line) technique was introduced
  - it is used to absorb surplus processor time by performing I/O transfers for other jobs

Types of Operating Systems

With the passage of time operating systems are divided into different categories. They can be categories as follows

- Single User OS
- Distributed OS
- Network OS
- Multiprocessor OS
- Real-time OS
- Embedded OS

Types of Operating Systems

Single User Operating Systems

- These types of OS were commonly used with computer systems referred as Personal Computers in 1980s, 1990s and early 2000s
- Cheaper OS
- Easy to use
- May run several different types of operating systems (Windows, MacOS, UNIX, Linux)
- Supports multiple applications
    - not dedicated for one type of application
- Have centralized resources
- Supports multimedia (images, text, graphics and audio/video)
- Example:
    - Windows 98,ME
    - Linux

Network Operating Systems are used to connect more than one computer

- User can login to a remote computer, to share its resources
- Network OS does not follow any transparency (e.g. location, migration)
    - Every operation has to be explicitly specified by the user
- User knows where his job is processing
- Examples:
    - Windows NT
    - Sun Solaris

- **Distribute computation** among several physical processors
- Follows the transparencies
- Requires networking infrastructure
    - Local area networks(LAN) & Wide area networks(WAN)
- May be either client-server or peer-to-peer systems
- Main features of Distributed System
    - Resource sharing
    - Computation speed up
    - Reliability
    - Communication

Types of Operating Systems

Embedded OS

These operating systems are dedicated for devices

These are normally built-in with devices

Example:

Mobile set OS

PDA OS (personal digital assistant, a handheld device that combines computing, telephone/fax, Internet and networking features.)

Issues:

Limited memory

Slow processors

Small display screens

Multiprocessor Operating system

- These are actually distributed operating systems which run on machines having more than one processor
- Communication is done via shared memory through message passing, pipes etc.

- Two types of **Distributed OS**
  - Tightly coupled DOS
    - Multiprocessor system
    - Processor share memory
      - Communication takes place through memory
  - Loosely coupled DOS
    - Single processor system
    - Clusters of workstations
      - Do not communicate via memory
      - Communicate via high speed buses, telephone lines
- Examples:
  - Windows 2000
  - Linux
  - Unix

Types of Operating Systems

- These OS run on **Real-time systems**
- Very fast operating system
- Normally designed for a set of specific applications
- A real-time system is any information processing system which has to respond to externally generated input stimuli within a finite and specified period
    - the correctness depends not only on the logical result but also the time it was delivered
    - failure to respond is as bad as the wrong response
- In a real time system emphasis is on timely execution of tasks than speed of execution
- Often used as a control device in a dedicated application such as controlling scientific experiments, medical imaging systems, industrial control systems, and some display systems
- Real-Time systems may be either hard or soft real-time
- Hard real-time system
    - systems where it is absolutely imperative that responses occur within the required deadline
    - E.g. Flight control systems
    - Secondary storage limited or absent, data stored in short term memory, or read-only memory (ROM)
- Soft real-time system
    - systems where deadlines are important but which will still function correctly if deadlines are occasionally missed
    - E.g. Data acquisition system
    - Limited utility in industrial control of robotics
    - Useful in applications (multimedia, virtual reality) requiring advanced operating-system features

Computing Environment

- **Traditional:** Stand-alone general purpose machines
- But blurred as most systems interconnect with others (i.e. the Internet)
- Portals provide web access to internal systems
- Network computers (thin clients) are like Web terminals
- Mobile computers interconnect via wireless networks
- Networking becoming ubiquitous – even home systems use firewalls to protect home computers from Internet attacks

Mobile: Handheld smartphones, tablets, etc
What is the functional difference between them and a “traditional” laptop?
Extra feature – more OS features (GPS, gyroscope)
Allows new types of apps like ***augmented reality***
Use IEEE 802.11 wireless, or cellular data networks for connectivity
Leaders are **Apple iOS** and **Google Android**

Distributed
Collection of separate, possibly heterogeneous, systems networked together
Network is a communications path, **TCP/IP** most common
Local Area Network (**LAN**)
Wide Area Network (**WAN**)
Metropolitan Area Network (**MAN**)
Personal Area Network (**PAN**)
Network Operating System provides features between systems across network
Communication scheme allows systems to exchange messages
Illusion of a single system

Computing Environment

Client-Server Computing

- Dumb terminals supplanted by smart PCs
- Many systems now servers, responding to requests generated by clients
  - Compute-server system provides an interface to client to request services (i.e., database)
  - File-server system provides interface for clients to store and retrieve files

Peer-to-Peer

Another model of distributed system

P2P does not distinguish clients and servers

- Instead all nodes are considered peers
- May each act as client, server or both
- Node must join P2P network
  - Registers its service with central lookup service on network, or
  - Broadcast request for service and respond to requests for service via ***discovery protocol***
- Examples include Napster and Gnutella, **Voice over IP** *(***VoIP***)* such as Skype

Computing Environment

Virtualization

Allows operating systems to run applications within other OSes

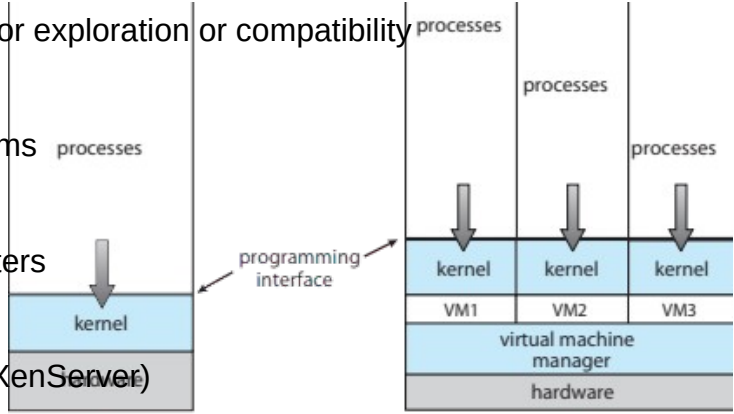
- Vast and growing industry

Emulation used when source CPU type different from target type (i.e. PowerPC to Intel x86)

- Generally slowest method
- When computer language not compiled to native code – **Interpretation**

Virtualization – OS natively compiled for CPU, running **guest** OSes also natively compiled

- Consider VMware running WinXP guests, each running applications, all on native WinXP **host** OS
- **VMM** provides virtualization services
- Use cases involve laptops and desktops running multiple OSes for exploration or compatibility
- Apple laptop running Mac OS X host, Windows as a guest
- Developing apps for multiple OSes without having multiple systems
- QA testing applications without having multiple systems
- Executing and managing compute environments within data centers
- VMM can run natively, in which case they are also the host
- There is no general purpose host then (VMware ESX and Citrix XenServer)

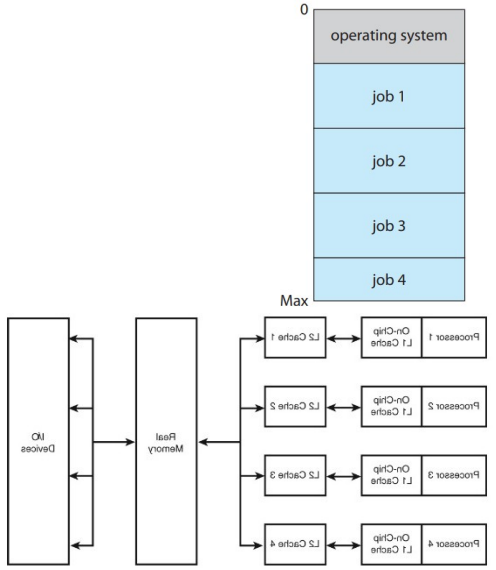


Operating System Services

- Operating systems provide an environment for execution of programs and services to programs and users
- A set of operating-system services provides functions that are helpful to the user
- **User interface**
  - Almost all operating systems have a user interface (**UI**)
    - Varies between **Command-Line (CLI)**, **Graphics User Interface (GUI)**, **Batch**
- **Program execution**
  - The system must be able to load a program into memory and to run that program, end execution, either normally or abnormally (indicating error)
- **I/O operations**
  - since user programs cannot execute I/O operations directly, the operating system must provide some means to perform I/O
- **File-system manipulation**
  - Programs need to read and write files and directories, create and delete them, search them, list file Information, permission management
  - Operating system helps the programs to achieve that
- **Communications**
  - Processes may exchange information, on the same computer or between computers over a network
  - Communications may be via shared memory or through message passing (packets moved by the OS)

Operating-System approach

Multiprogramming
The idea is as follows: The operating system keeps several jobs in memory simultaneously Since, in general, main memory is too small to accommodate all jobs, the jobs are kept initially on the disk in the job pool. This pool consists of all processes residing on disk awaiting allocation of main memory.



Multiprocessing
There are several different strategies for designing a computer system. Single-processor systems have only one processor, while multiprocessor systems contain two or more processors that share physical memory and peripheral devices.

Multitasking
Multitasking is when multiple jobs are executed by the CPU simultaneously by switching between them. Switches occur so frequently that the users may interact with each program while it is running. Multitasking about time sharing

Multithreading
Multitasking is when multiple jobs are executed by the CPU simultaneously by switching between them. Switches occur so frequently that the users may interact with each program while it is running. An OS does the following activities related to multitasking

Problems because of Multiprogrammin

- Memory Protection
- I/O Protection
- CPU Protection

Solution of Multiprogramming Problems

Memory Protection (Solution)

- Every Program residing in memory is assigned a limit
- If a program exceed its limit in user mode, it is trapped

I/O Protection Solution

- I/O instructions can’t be used in user mode
- Issuing I/O instructions in user-mode generate a trap

CPU Protection (Solution)

- Time, a controller, generates interrupts at fixed intervals.
- Interrupts transfer control to kernel in kernel mode.

Operating System Services

Error detection

OS needs to be constantly aware of possible errors

May occur in the CPU and memory hardware, in I/O devices, in user program

For each type of error, OS should take the appropriate action to ensure correct and consistent computing

Debugging facilities can greatly enhance the user's and programmer's abilities to efficiently use the system

Resource allocation

When multiple users or multiple jobs running concurrently, resources must be allocated to each of them

Many types of resources - Some (such as CPU cycles, main memory, and file storage) may have special allocation code, others (such as I/O devices) may have general request and release code

Accounting

To keep track of which users use how much and what kinds of computer resources

Protection and security

The owners of information stored in a multiuser or networked computer system may want to control use of that information, concurrent processes should not interfere with each other

Protection

involves ensuring that all access to system resources is controlled

Security

of the system from outsiders requires user authentication, extends to defending external I/O devices from invalid access attempts

If a system is to be protected and secure, precautions must be instituted throughout it. A chain is only as strong as its weakest link

OS Management

Process Management

A process is a program in execution

A process needs certain resources, including CPU time, memory, files, and I/O devices, to accomplish its task

The operating system is responsible for the following activities in connection with process manager

- Process creation and termination
- Process suspension and resumption
- Provision of mechanisms for:
  - process synchronization
  - process communication

Memory Management

Memory is a large array of words or bytes, each with its own address

It is a repository of quickly accessible data shared by the CPU and I/O devices.

Main memory is a volatile storage device

- It looses its contents in the case of system failure

The operating system is responsible for the following activities in connections with memory management:

- Keep track of which parts of memory are currently being used and by whom
- Decide which processes to load when memory space becomes available
- Allocate and deallocate memory space as needed

I/O System Management

Operating system is responsible for I/O system management The I/O system consists of:

- A buffer-caching system
- A general device-driver interface
- Drivers for specific hardware devices

File Management

- file is a collection of related information defined by its creator
- Commonly, files represent programs (both source and object forms) and data
- The operating system is responsible for the following activities in connections with file management:
  - –File creation and deletion
  - –Directory creation and deletion
  - –Support of primitives for manipulating files and directories
  - –Mapping files onto secondary storage
  - –File backup on stable (nonvolatile) storage media

Protection refers to a mechanism for controlling access by programs, processes, or users to both system and user resources

The protection mechanism must:

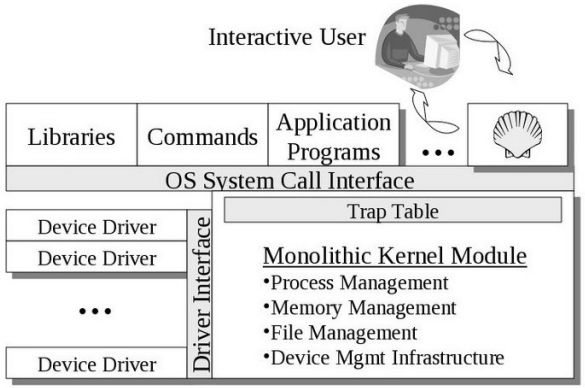
- distinguish between authorized and unauthorized access
- specify the controls to be imposed

OS Structure

- Kernel is part of operating system that includes most heavily used functions
- Generally kernel is permanently in main memory
- A privileged, nucleus or supervisory mode into which the machine switches when the OS is running
- Enables the execution of such privileged instructions
  - Such as in the area of I/O operations, which are not available to user mode

Monolithic Approach

- Functionality of OS is invoked through simple function call which is one large program
- Device drivers are loaded into kernel and become part of Operating system



A monolithic kernel, such as Linux and other Unix systems.

1.8.4. Microkernels

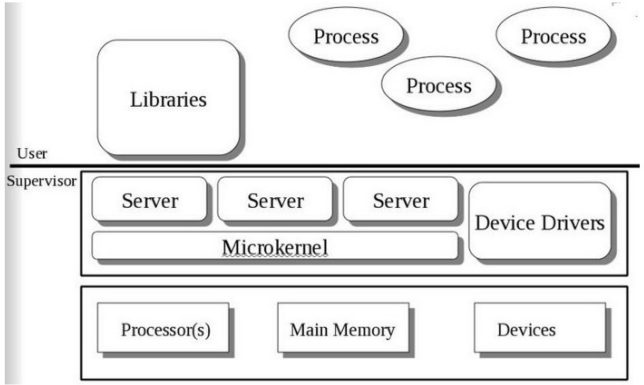
This structures the operating system by removing all nonessential portions of the kernel and implementing them as system and user level programs.

- Generally they provide minimal process and memory management, and a communications facility.
- Communication between components of the OS is provided by message passing.

The *benefits* of the microkernel are as follows:

- Extending the operating system becomes much easier.
- Any changes to the kernel tend to be fewer, since the kernel is smaller.
- The microkernel also provides more security and reliability.

Main *disadvantage* is poor performance due to increased system overhead from message passing.



OS Structure

1. Scheduling classes
2. File systems
3. Loadable system calls
4. Executable formats
5. STREAMS modules
6. Miscellaneous
7. Device and bus drivers

1.8.3. Layered Approach

This approach breaks up the operating system into different layers.

- This allows implementers to change the inner workings, and increases modularity.
- As long as the external interface of the routines don't change, developers have more freedom to change the inner workings of the routines.
- With the layered approach, the bottom layer is the hardware, while the highest layer is the user interface.
    - The main *advantage* is simplicity of construction and debugging.
    - The main *difficulty* is defining the various layers.
    - The main *disadvantage* is that the OS tends to be less efficient than other implementations.

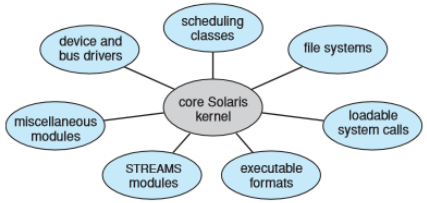


Figure 2.15 Solaris loadable modules.

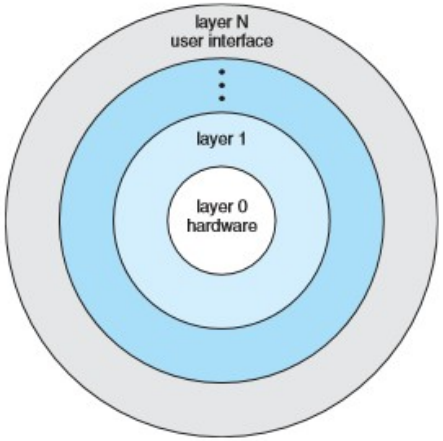


Figure 2.13 A layered operating system.

Hybrid Systems

Linux and Solaris are monolithic and modular (Loadable kernel modules)
Windows are primarily monolithic but supports microkernel approach with loadable kernel modules.

Dual Mode

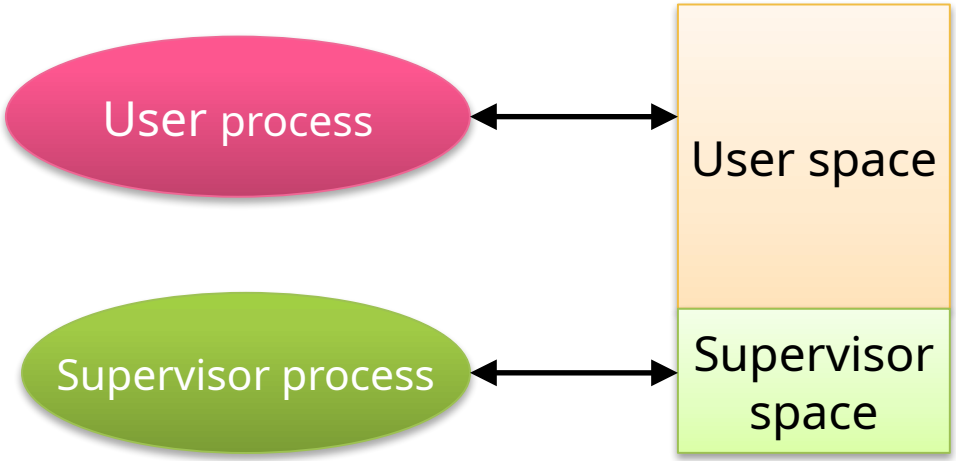
At the very least, we need two separate modes of operation:

- **user mode**
- **kernel mode (also called supervisor mode, system mode, or privileged mode).**

A bit, called the mode bit, is added to the hardware of the computer to indicate the current mode: kernel (0) or user (1).

Kernel Mode

- The techniques by which a user program requests the kernel's services are;
  - System calls
  - Message passing
- The process can only be switched into kernel mode by the above actions
- No other way that a user program can execute kernel instructions
- This facility is not given to user program in order to enforce system security



Supervisor & user memory

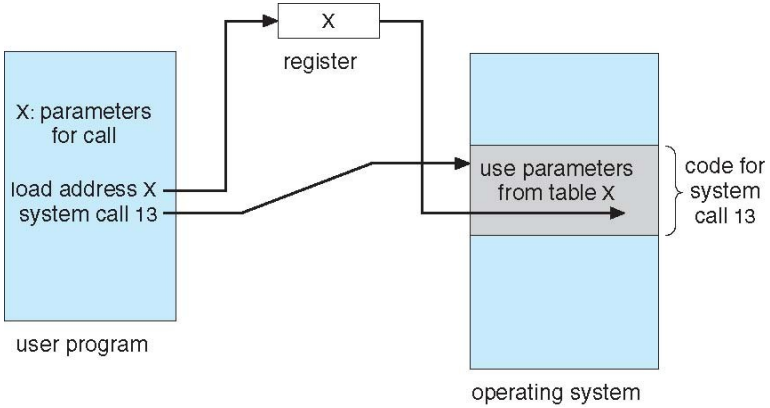
System Calls

- User program communicate with OS and request its services by making system calls
- Corresponding to each system call is a library procedure that user program can call
- System calls provide the interface between a running program and the OS
- These calls are generally available as assembly language instruction
- All instruction with the hardware is implemented by system calls
- OS provides an additional layer of subroutines called an API (application programming interface), between programmer and system call interface
- To compile a program, a process calls the command interpreter or shell, reads command from the terminal
- Shell will create a new process to compile the program
- When the process has completed compilation, it executes a system call to terminate itself
- Other process system calls are also available to;
    - Request for more memory
    - Released unused/extra memory
    - Process and memory management
- System calls related to file systems;
    - System calls are needed to create, read, move and write a file
    - A file should be opened before read it
    - Similarly, after reading file must be closed which is done by system calls
    - System calls are required to
        - create and remove a directory
        - manipulate file with in different directories
- System V Unix provides about 64 system calls e.g.
    - create(name, amode)
    - open(name, oflag, amode)
    - close(fd)
- MS-DOS uses software interrupts similar to system calls

System Call

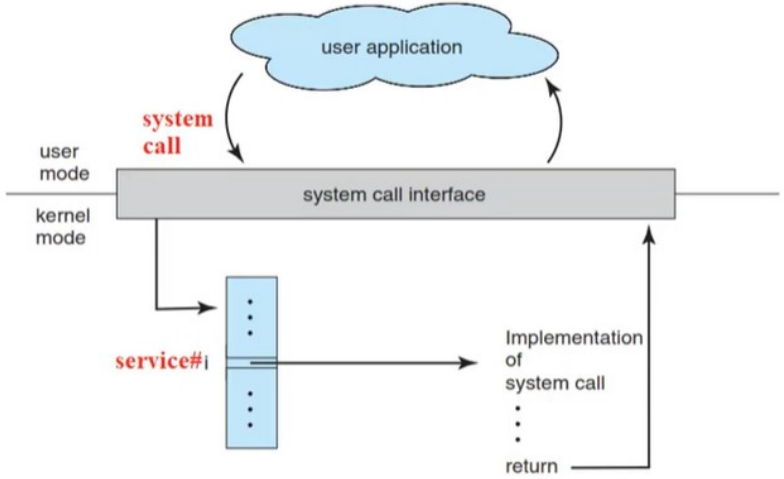
Three general methods are used to pass parameters between a running program and the operating system

- Pass parameters in registers
- Store the parameters in a table in memory, and the table address is passed as a parameter in a register
- Push (store) the parameters onto the stack by the program, and pop off the stack by operating system



Types of System Calls

- Process control
- File management
- Device management
- Information maintenance
- Communications



Lab

- Virtual Machine Installation
- Ubuntu installation
- Commands of Ubuntu



Thank you